

JAYS SHOP

Digital Retail & In-Stadium Fulfillment Platform

VERSION 2.0

COMPREHENSIVE PROJECT DOCUMENTATION

A Complete Business & Technical Reference for Executives, Management,
Developers, Designers & Future Stakeholders

Prepared for Store Operations, Engineering, Retail Management & Executive Leadership

Table of Contents

1. Executive Summary	5
2. Project Vision and Objectives	5
Core Objectives	5
3. Business Problem and Solution Overview	5
The Business Problem	5
The Platform Solution	5
4. Business Value and Strategic Impact	6
5. Customer Experience Journey	6
6. Operational Workflow Optimization	7
7. Functional Requirements	7
8. Non-Functional Requirements	8
9. Complete System Architecture	9
Architectural Principles	9
10. Technology Stack	9
11. Database Design and Entity Relationships	10
Key Relationships	11
12. Application Modules	11
13. Screen-by-Screen Documentation	11
Landing Page (/)	11
Shop Catalog (/shop)	12
Product Detail Page (/shop/[slug])	12
Popular Players (/players, /players/[slug])	12
Digital Hold Confirmation (/holds/[reservationId])	13

My Holds (/my-holds)	13
Admin Dashboard (/admin)	13
Admin Holds Queue (/admin/holds)	13
Admin Products (/admin/products)	14
Admin Game Days (/admin/game-days)	14
Admin Analytics & Reports (/admin/analytics, /admin/reports)	14
14. User Roles and Permissions	16
Role Summary	16
15. Customer Workflow	16
16. Staff Workflow	17
17. Admin Workflow	17
18. Inventory Management Process	17
19. Digital Hold Lifecycle	18
Creation	18
Resolution	18
Expiry	19
20. Game Day Operations	19
21. Product Management	19
22. Sales & Clearance Management	19
23. Brand and Category Management	21
24. Reporting and Analytics	21
25. Security and Access Control	21
26. Performance and Scalability Considerations	22
27. API and Data Flow Architecture	22

28. Business Rules and Validation Logic	23
29. UI/UX Design Principles	23
30. Error Handling Strategy	23
31. Future Enhancements and Version Roadmap	24
32. Testing Strategy	24
33. Deployment Architecture	24
34. Maintenance and Support Plan	25
35. Project Risks and Mitigation Strategies	25
36. Conclusion and Long-Term Vision	25

1. Executive Summary

Jays Shop is a full-stack digital retail and in-stadium fulfillment platform built for a Toronto Blue Jays fan shop operation. It unifies a mobile-first product browsing & reservation experience, a real-time multi-location inventory ledger, a proprietary Digital Hold reservation system, point-of-sale integration, and a complete staff/management back office into a single connected system. Version 2.0 reflects the platform's evolution from an initial hold-and-pickup concept into a mature retail operations platform spanning product merchandising, player-driven cross-selling, sales & clearance management, game-day operational rules, and management-grade reporting.

36 Documented functional areas & screens	4 Role-based permission tiers	20+ Admin operational & reporting API endpoints	100% Shared inventory ledger across web, hold & POS
---	---	--	--

This Version 2.0 documentation is the authoritative reference for how the platform is built, how it behaves, and why it was designed this way — intended for executives evaluating business value, engineers extending the system, designers maintaining UI consistency, and operators running day-to-day retail activity.

2. Project Vision and Objectives

The platform's vision is to remove the historic trade-off between digital browsing convenience and in-person pickup certainty. Fans should be able to discover, evaluate, and reserve merchandise from anywhere, then complete a fast, trustworthy pickup at the stadium — without needing an account, without guessing at stock, and without waiting in an undifferentiated line.

Core Objectives

- Provide a mobile-first, frictionless storefront for browsing and reserving Blue Jays merchandise.
- Guarantee reservation accuracy through a single, real-time, size- and location-level inventory ledger.
- Give staff guided, policy-enforced tools for fulfilling holds, managing stock, and resolving customer questions.
- Give management continuous, actionable visibility into sales, holds, conversion, and inventory health.
- Build a foundation that scales cleanly across additional stadium sections, product lines, and future venues.

3. Business Problem and Solution Overview

The Business Problem

- Walk-in-only stadium retail forces fans into an undifferentiated queue with no visibility into stock.
- Game-day demand surges strain staff and create inconsistent pickup experiences across counters and sections.
- Manual inventory processes make it difficult for management to know true stock health across locations.

The Platform Solution

- A **Digital Hold** system lets fans reserve a specific product, size, and quantity against a live inventory count, producing an instant reservation code and expiration window.
- A unified **SizeInventory** ledger, shared by the storefront, the hold engine, and point-of-sale transactions, ensures one accurate source of truth at all times.
- Two dedicated pickup channels — a standard Gate 5 counter and an express Stadium Queue (Section 123) — route fans appropriately, with the express channel automatically enabled only on active game days.
- A role-based admin back office (STAFF / MANAGER / OWNER, plus VIEWER) gives every level of the organization exactly the tools and visibility they need.

4. Business Value and Strategic Impact

Stakeholder	Value Delivered
Customers / Fans	Confidence that a reserved item will be available; flexible pickup options; no account required; instant self-service order lookup by phone.
Retail Associates	A single guided queue for fulfilling holds; automatic policy enforcement; clear urgency indicators for expiring holds.
Supervisors / Managers	Real-time oversight of active holds and inventory; ability to manage exceptions confidently; complete audit trail for every action.
Executives / Management	Consolidated conversion, revenue, and inventory reporting; a scalable platform that grows with the retail program.

Strategically, the platform strengthens retail as a genuine extension of the game-day experience — a touchpoint where the same standard of excellence fans expect from Rogers Centre itself is reflected in how effortlessly they can find, reserve, and collect merchandise.

5. Customer Experience Journey



Figure 5.1 — End-to-end customer experience journey

- **Discover:** Mobile-first catalog with category, brand, and audience filtering, plus a dedicated Popular Players hub linking fans to player-specific gear.
- **Evaluate:** Product detail pages show real-time size availability, related products, target-audience labeling, and customer reviews.
- **Reserve:** The Digital Hold flow reserves a specific size and quantity against live stock, returning a reservation code and countdown.
- **Confirm:** A confirmation page with QR code and clear pickup instructions removes any ambiguity about the next step.
- **Pick Up:** Fans choose Gate 5 (standard) or the Stadium Queue (Section 123, game days only) based on their location and timing.

- **Re-engage:** Favorites, phone-based order lookup, and review submission keep the relationship active beyond a single visit.

6. Operational Workflow Optimization

Every operational workflow in the platform is designed to keep customer service, inventory, fulfillment, and pickup continuously synchronized, so an action in one area is immediately and correctly reflected everywhere else.

- **Customer service:** phone-number lookup resolves any fan's active/past holds instantly, with no account or escalation needed.
- **Inventory:** every hold, sale, return, and transfer writes to the same append-only ledger (*InventoryTransaction*), eliminating drift between channels.
- **Fulfillment:** a single Holds queue with urgency indicators and partial-fulfillment support minimizes staff decision time per transaction.
- **Pickup:** printable receipts and clear channel routing keep the physical handoff fast and consistent regardless of which staff member is on duty.

7. Functional Requirements

ID	Requirement
FR-1	Product catalog browsing with category/subcategory/brand/audience filters and search
FR-2	Product detail view with real-time size-level availability
FR-3	Digital Hold creation with phone-based customer identification (no account required)
FR-4	Hold precheck endpoint to preview eligibility and effective hold duration before commit
FR-5	Self-service hold status lookup, pickup, and release via reservation code
FR-6	Customer hold history lookup by phone number ("My Holds")
FR-7	Popular Players directory with linked shoppable gear (PlayerProduct mapping)
FR-8	Product likes (heart) and client-side Favorites wishlist
FR-9	Customer product reviews (rating + comment) with admin moderation
FR-10	Site feedback submission with admin inbox and delete capability
FR-11	Admin product CRUD with multi-image upload and size-inventory management
FR-12	Admin hold queue with resolve (full/partial pickup, release) actions
FR-13	Admin hold-settings configuration (standard/extended hold hours, 48-hour toggle)
FR-14	Admin game-day calendar management (date, start time, opponent, note)
FR-15	Admin inventory transfer between locations with full transaction logging
FR-16	Admin brand and player management (CRUD)
FR-17	Admin analytics overview and sales/hold reporting dashboards with export

ID	Requirement
FR-18	Admin user management with role assignment (OWNER only)
FR-19	Audit log of all sensitive admin and system actions
FR-20	POS transaction webhook ingestion with idempotency and API-key auth
FR-21	Automated hold expiry via scheduled cron job
FR-22	Slack notifications for new holds and expirations, with interactive resolution

8. Non-Functional Requirements

Quality Attribute	Requirement Detail
Reliability	Inventory transactions are wrapped in database transactions to prevent overselling or double-counting.
Availability	Hold expiry and inventory sync run automatically (cron + inline auto-expire) so data never goes stale between visits.
Security	Role-based access control (VIEWER < STAFF < MANAGER < OWNER) enforced on every admin route; bcrypt password hashing; JWT sessions with 8-hour expiry.
Auditability	Every sensitive state change (holds, inventory, products, admins, settings) is recorded to an immutable AuditLog with before/after snapshots.
Performance	Admin analytics responses are cached briefly (in-memory, ~30s) to absorb dashboard polling load without stressing the database.
Scalability	Multi-tenant-aware schema (tenantId on core models) and a multi-location inventory model support expansion to new venues or storefronts.
Usability	Mobile-first responsive design across public storefront and admin dashboard; no customer account/password required.
Maintainability	Consistent Next.js App Router conventions, typed Prisma schema, and centralized constants/taxonomy files reduce change risk.
Rate Limiting	Admin API paths are protected by a sliding-window rate limiter (default 100 requests/min) to guard against abuse.
Data Integrity	Idempotent POS event ingestion (externalId uniqueness) prevents duplicate transaction application.

9. Complete System Architecture

Jays Shop is built as a single Next.js 14 App Router application, serving both the public storefront and the admin dashboard from one codebase, backed by a PostgreSQL database via Prisma ORM. Public and admin surfaces are fully separated at the routing level via route groups, while sharing the same data layer, taxonomy constants, and audit/notification infrastructure.

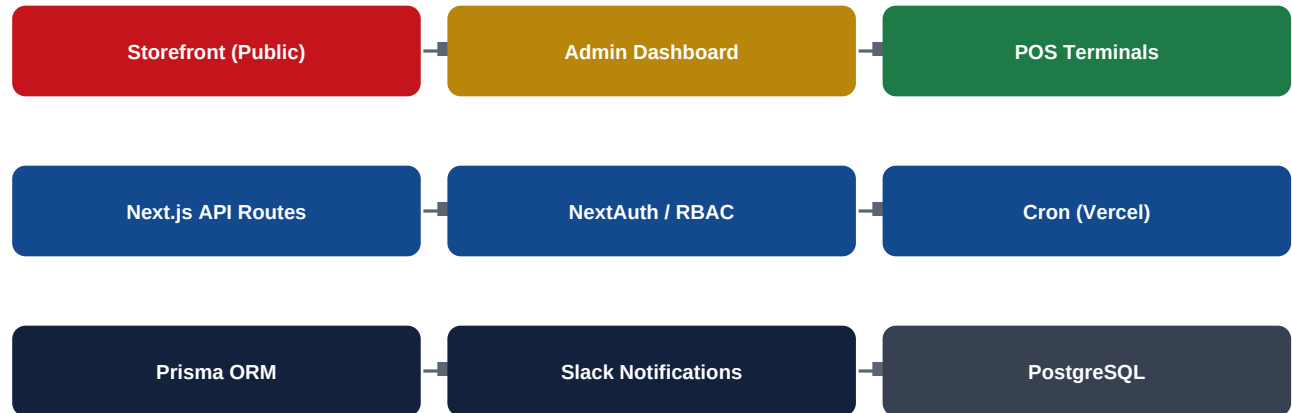


Figure 9.1 — High-level system architecture layers

Architectural Principles

- **Single source of truth:** one PostgreSQL database and Prisma schema back every channel — web, hold engine, and POS.
- **Server-rendered + API-driven:** Next.js App Router server components handle SEO-friendly pages; client components handle interactivity (cart, favorites, hold flows).
- **Route-group isolation:** ``(public)`` and ``(admin)`` route groups keep customer and staff experiences cleanly separated while sharing one deployment.
- **Middleware-enforced security:** all ``/admin`` pages and ``/api/admin/*`` routes pass through NextAuth-aware middleware before reaching business logic.
- **Transactional integrity:** hold creation/resolution and inventory transfers execute inside Prisma database transactions to guarantee consistency.
- **Event/audit trail:** every state-changing action is captured to the AuditLog model, independent of which channel initiated it.

10. Technology Stack

Layer	Technology
Framework	Next.js 14.2 (App Router), React 18, TypeScript 5
Database / ORM	PostgreSQL, Prisma 5.22 (@prisma/client)
Authentication	NextAuth 4.24 — Credentials provider, JWT sessions (8-hour expiry)

Layer	Technology
Styling / UI	Tailwind CSS 3.4, Radix UI primitives, lucide-react icons, sonner toasts, vaul drawers
Charts	recharts (admin analytics & reporting dashboards)
Media	Local /public/uploads file storage, Supabase JS client, next-pwa (Cloudinary image caching)
Security	bcryptjs password hashing, nanoid ID generation
AI Assistant	OpenAI 4.x — powers the “Birdie” chat assistant (/api/chat)
QR / Receipts	qrcode.react for hold confirmation QR codes
Dev Tooling	tsx (scripts/seed runner), ESLint (eslint-config-next), tsc --noEmit typecheck
Deployment	Vercel — daily cron (0 0 * * *) triggers hold expiry; prisma generate on build & postinstall

11. Database Design and Entity Relationships

The schema is organized around three functional clusters: **commerce** (Product, Brand, Player/PlayerProduct, ProductReview, ProductLike), **inventory & fulfillment** (SizeInventory, StoreLocation, Hold, HoldHistory, SalesHistory, InventoryTransaction), and **platform/operations** (Tenant, Admin, AuditLog, GameDay, HoldSettings, SlackSettings, PosApiKey, PosEvent, SiteFeedback).

Tenant Multi-tenant root	Product Catalog item	Brand Brand metadata
Player Player profile	PlayerProduct Player ↔ Product join	Customer Phone-identified fan
Hold Reservation record	SizeInventory Per-size, per-location stock	StoreLocation Physical/queue location
InventoryTransaction Append-only stock ledger	HoldHistory Resolved-hold snapshot	SalesHistory Completed-sale record
Admin Staff account	AuditLog Immutable action log	GameDay Scheduled home game
HoldSettings Hold-duration config	SlackSettings Notification config	PosApiKey POS credential
PosEvent POS transaction log	ProductReview Customer review	ProductLike Session-based heart/like
SiteFeedback Public feedback message		

Key Relationships

- **Product 1–N SizeInventory** — each product's stock is tracked per size, per StoreLocation.
- **Product 1–N Hold, Customer 1–N Hold** — a hold links a specific product/size reservation to a phone-identified customer.
- **Hold 1–1 HoldHistory**, optional **1–1 SalesHistory** — resolution always snapshots history; a sale record is created only when items are picked up.
- **Player N–N Product** via **PlayerProduct** — each mapping carries a display label (e.g. “Home Jersey”) and sort order for the player detail page.
- **StoreLocation 1–N InventoryTransaction** (from/to) — every stock movement references its source and destination location.
- **Admin 1–N AuditLog** — every admin-initiated action is traceable back to the acting user.
- **PosApiKey 1–N PosEvent** — every POS transaction is tied to the credential that submitted it, enabling per-terminal auditing.

12. Application Modules

Module	Responsibility
Storefront	Catalog browsing, product detail, brands, players, cart, account, informational pages
Digital Hold Engine	Hold creation, precheck, resolution, expiry, history/sales archival
Inventory Management	Size/location stock tracking, transfers, restocks, transaction ledger
Point-of-Sale Integration	API-key authenticated webhook ingestion, idempotent transaction application
Player & Brand Merchandising	Player profiles, linked shoppable gear, brand directory
Engagement	Product likes, client-side favorites, customer reviews, site feedback
Admin Operations	Product/inventory CRUD, hold queue management, game-day calendar
Reporting & Analytics	Operational overview dashboard, sales/conversion reports, likes analytics, exports
Identity & Access	NextAuth admin login, role-based authorization, admin user management
Notifications	Slack new-hold and expiry alerts, interactive Slack resolution
Governance	Audit logging across all sensitive actions, orphaned-media audit tooling

13. Screen-by-Screen Documentation

Each screen below is documented with its purpose, primary objective, user actions, inputs and outputs, and how it connects to the rest of the platform.

Landing Page (I)

Attribute	Detail
Purpose	First impression and orientation hub for fans arriving at the site.

Attribute	Detail
Primary Objective	Surface featured merchandise and any upcoming/active home game.
User Actions	Browse featured products; view Upcoming Match card when a home game is scheduled.
Inputs / Outputs	In: none. Out: navigation to Shop, Players, Brands.
Navigates To	/shop, /players, /brands
Business Value	Drives initial engagement and highlights game-day relevance without cluttering the page when no game is scheduled.

Shop Catalog (/shop)

Attribute	Detail
Purpose	Primary product discovery and filtering surface.
Primary Objective	Let fans find the right product quickly via category, subcategory, brand, and audience filters.
User Actions	Filter by category/brand/audience; search; view Sales & Clearance and Blank Jersey collections; optional player-bundle view via productIds query.
Inputs / Outputs	In: filter/search query params. Out: product grid, navigation to PDP.
Navigates To	/shop/[slug]
Business Value	Central conversion surface; consistent target-audience labeling and clearance subcategories reduce browsing friction.

Product Detail Page (/shop/[slug])

Attribute	Detail
Purpose	Deep-dive product page supporting the purchase/reserve decision.
Primary Objective	Give fans everything needed to decide with confidence and start a hold.
User Actions	View images, price, size availability, target audience/color, reviews; like/favorite; start a Digital Hold.
Inputs / Outputs	In: slug param. Out: hold creation request, review submission.
Navigates To	/holds/[reservationId] (after hold), related PDPs
Business Value	Highest-intent screen; real-time availability and reviews increase reservation conversion.

Popular Players (/players, /players/[slug])

Attribute	Detail
Purpose	Player-driven merchandising hub linking fans to associated shoppable gear.
Primary Objective	Cross-sell jerseys and gear through fan affinity for specific players.
User Actions	Browse/search players by name or position; view player bio, stats, and linked gear; jump to PDP or a filtered shop bundle.
Inputs / Outputs	In: search/position filters, slug. Out: navigation to /shop/[slug] or /shop?productIds=.

Attribute	Detail
Navigates To	/shop, /shop/[slug]
Business Value	Differentiated merchandising angle that increases basket relevance for fans of specific players.

Digital Hold Confirmation (/holds/[reservationId])

Attribute	Detail
Purpose	Post-reservation confirmation and status screen.
Primary Objective	Give the fan a clear, shareable record of their reservation.
User Actions	View reservation code, countdown, QR code, pickup instructions; self-service pickup/release.
Inputs / Outputs	In: reservationId. Out: PATCH action (pickup/release).
Navigates To	/my-holds
Business Value	Removes uncertainty after reservation; QR code speeds up in-person pickup verification.

My Holds (/my-holds)

Attribute	Detail
Purpose	Self-service lookup of a customer's hold history.
Primary Objective	Let a returning fan check all active/past holds without an account.
User Actions	Enter phone number; view list of holds and statuses.
Inputs / Outputs	In: phone number. Out: list of Hold records.
Navigates To	/holds/[reservationId]
Business Value	Removes dependency on staff for basic status questions, reducing service load.

Admin Dashboard (/admin)

Attribute	Detail
Purpose	Landing screen for staff after login; operational at-a-glance view.
Primary Objective	Orient staff toward the highest-priority tasks for the day.
User Actions	Review key metrics; navigate to Holds, Inventory, Reports.
Inputs / Outputs	In: session/role. Out: navigation.
Navigates To	/admin/holds, /admin/products, /admin/analytics
Business Value	Reduces time-to-first-action for staff starting a shift.

Admin Holds Queue (/admin/holds)

Attribute	Detail
Purpose	Operational queue for fulfilling and resolving active reservations.
Primary Objective	Let staff resolve holds accurately and quickly, including partial fulfillment.
User Actions	View active holds with urgency indicators; resolve as picked-up, partial, or released.
Inputs / Outputs	In: holdId, action, fulfilledQty. Out: updated Hold/HoldHistory/SalesHistory records.
Navigates To	(print)/admin/holds/[id]/print
Business Value	Directly reduces staff cognitive load and pickup handling time on busy game days.

Admin Products (/admin/products)

Attribute	Detail
Purpose	Product catalog and inventory management.
Primary Objective	Maintain accurate product data, imagery, taxonomy, and stock.
User Actions	Create/edit/archive products; upload images; manage size inventory; assign category flags (Sale, Clearance, Blank Jersey, etc.).
Inputs / Outputs	In: product form data, images. Out: Product/SizeInventory records.
Navigates To	/admin/inventory/history
Business Value	Keeps the storefront's product data and stock accurate at the source.

Admin Game Days (/admin/game-days)

Attribute	Detail
Purpose	Scheduling interface for upcoming home games.
Primary Objective	Keep the single source of truth for game-day-dependent business rules and landing-page display in sync.
User Actions	Add/delete game days with date, start time, opponent, note.
Inputs / Outputs	In: date, startTime, opponent, note. Out: GameDay record.
Navigates To	Reflected automatically on landing page and hold-eligibility logic.
Business Value	Ensures Section 123 hold eligibility and landing-page matchup display are always correct and current.

Admin Analytics & Reports (/admin/analytics, /admin/reports)

Attribute	Detail
Purpose	Management-facing operational and business intelligence dashboards.
Primary Objective	Give supervisors and executives continuous visibility into inventory health and sales performance.
User Actions	View inventory breakdowns, conversion/no-show rates, top products, weekly revenue; export data.
Inputs / Outputs	In: date range/period filter. Out: aggregated metrics, CSV export.
Navigates To	N/A (terminal reporting screen)

Attribute	Detail
Business Value	Converts operational data into planning and staffing insight without manual reconciliation.

14. User Roles and Permissions

Access control is enforced via a strict role hierarchy: **VIEWER** < **STAFF** < **MANAGER** < **OWNER**. Every gated admin action declares a minimum required role in a central permission map (*actionMinRole*), and a single *requireRole()* check is applied consistently at the top of every protected API route.

Action	Minimum Role
inventory:read	STAFF
inventory:write	MANAGER
holds:read	STAFF
holds:resolve	MANAGER
transfers:create	MANAGER
pos-keys:manage	OWNER
game-days:manage	MANAGER
hold-settings:manage	MANAGER
analytics:read	STAFF
players:read	STAFF
players:write	MANAGER
brands:read	STAFF
brands:write	MANAGER
feedback:read	STAFF
feedback:write	MANAGER
admin:manage	OWNER
tenant:manage	OWNER

Role Summary

- **VIEWER** — read-only access baseline; no write permissions granted by default.
- **STAFF** — day-to-day floor operations: view inventory, holds, analytics, players, brands, feedback.
- **MANAGER** — adds write access: resolve holds, edit inventory/products, manage game days, hold settings, players, and brands.
- **OWNER** — full control: admin user management, POS API key issuance, tenant management, and every MANAGER/STAFF capability.

15. Customer Workflow



Figure 15.1 — Customer digital-hold workflow

A fan browses `/shop` or `/players`, opens a product detail page, selects a size/quantity, and submits a phone number to place a hold via `POST /api/holds`. The system validates real-time availability, computes the effective hold window (standard vs. extended vs. game-day-adjusted), and returns a reservation code with an expiration timestamp. The fan can look up status anytime via reservation code or phone number, and self-service pickup/release the hold from the confirmation page before staff involvement is even required.

16. Staff Workflow



Figure 16.1 — Retail associate operational workflow

Staff authenticate via the NextAuth-protected `/admin/login`, then work primarily from the `/admin/holds` queue — resolving holds as full pickups, partial pickups, or releases with a single action. Inventory adjustments, restocks, and transfers are performed from `/admin/products` and `/admin/inventory/history`, with every change automatically written to the InventoryTransaction ledger. Staff with only STAFF-level role can read all operational data but must escalate write actions requiring MANAGER approval (e.g., inventory edits, hold-settings changes).

17. Admin Workflow



Figure 17.1 — Manager/Owner administrative workflow

MANAGER and OWNER roles configure the operational rules that STAFF work within: game-day scheduling (`/admin/game-days`), hold-duration policy (`/admin/hold-settings`), and full catalog, brand, and player management. OWNER additionally manages admin accounts and roles (`/admin/admins`), issues POS API keys (`/admin/pos-keys`), and reviews the full audit trail (`/admin/audit-log`) for governance and compliance oversight.

18. Inventory Management Process

Inventory is tracked at the most granular level relevant to retail decisions: **product × size × location** (the `SizeInventory` model), with `Product` maintaining aggregate mirror totals (`quantity`, `heldQuantity`, `pickedQuantity`) for fast catalog display.

- **Restock:** `POST /api/admin/products/[id]/restock` increases available quantity at a location and logs an `InventoryTransaction` (type *adjustment*).
- **Transfer:** `POST /api/admin/inventory/transfer` moves stock between two `StoreLocation` rows, logging *transfer* transactions referencing both source and destination.
- **Hold reservation:** placing a hold increments `heldQuantity` at the fulfilling location (type *hold-reserve*); releasing or expiring decrements it (type *hold-release*).
- **POS sale/return:** POS webhook events increment `pickedQuantity`/adjust `quantity` directly (types *sale/return*), keeping in-person transactions reconciled with the same ledger.
- **Status rollup:** product `status` (AVAILABLE/SOLD/ARCHIVED) is automatically recalculated from aggregate size-inventory availability after every change.

19. Digital Hold Lifecycle

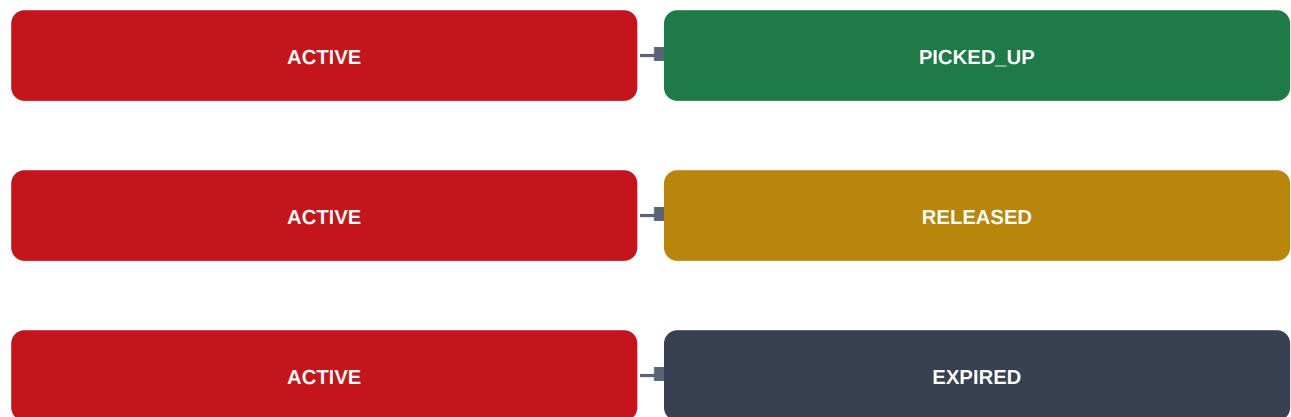


Figure 19.1 — Hold status transitions

Creation

- Upserts the `Customer` by phone number; enforces a maximum of **3 active holds** per customer.
- Determines pickup channel: standard (Gate 5) vs. stadium (Section 123 queue) — stadium pickup requires an active game day, or the request is rejected (`SECTION_123_GAME_DAY_ONLY`).
- Computes the effective hold window via `getEffectiveHoldHours()`: stadium holds always use the standard window; standard holds use the extended 48-hour window only when enabled and it is not a game day.
- Runs inside a database transaction: re-validates live availability, increments `heldQuantity`, logs an `InventoryTransaction`, creates the `Hold` row, syncs product aggregates, and writes an `AuditLog` entry (`hold.created`).

Resolution

- `resolveHold()` accepts a target status (`PICKED_UP`/`RELEASED`/`EXPIRED`) and an optional partial fulfillment quantity.

- Partial pickups mark the original hold ``PICKED_UP`` for the fulfilled portion and spin off a new ``ACTIVE`` hold for the remainder, inheriting the original expiration.
- A ``HoldHistory`` snapshot is always written; a ``SalesHistory`` row is written only when items are actually picked up.

Expiry

- ``expireAllOverdueHolds()`` runs via the daily Vercel cron job (``/api/cron/expire-holds``, Bearer ``CRON_SECRET`` auth) and sends Slack expiry notifications.
- ``autoExpireOverdueHolds()`` runs inline on read endpoints (hold lookup, admin analytics) so no client ever sees a stale `ACTIVE` hold, even between scheduled cron runs.

20. Game Day Operations

The ``GameDay`` model (``date``, ``startTime``, ``opponent``, ``note``) is the single source of truth for all game-day-dependent behavior across the platform — eliminating any risk of mismatch between what staff schedule and what fans see.

- **Admin scheduling:** ``/admin/game-days`` lets `MANAGER+` create/delete entries, validated for correct date and ``HH:mm`` time format, with duplicate-date protection and full audit logging.
- **Hold eligibility:** ``isGameDay()`` performs a UTC-normalized exact-date lookup used by the hold engine to gate Section 123 stadium-queue eligibility and to select standard vs. extended hold windows.
- **Landing page display:** ``GET /api/game-days/next`` returns the earliest upcoming home game; ``UpcomingMatchCard`` renders the Blue Jays vs. opponent matchup (with logos), formatted date and start time — all in UTC to prevent off-by-one-day timezone bugs — and renders nothing at all when no game is scheduled, avoiding empty banners.

21. Product Management

Products are the platform's core commerce entity, supporting rich taxonomy, multi-image merchandising, and per-size/location inventory. Admin product management covers the full lifecycle from creation through archival.

- **Creation/editing:** ``/admin/products`` supports multi-image upload, category/subcategory/brand/color assignment, and toggling of merchandising flags (``isFeatured``, ``isClearance``, ``isSport``, ``isBlankJersey``, ``isNewArrival``, ``isWorldSeries``, ``isAuthenticated``, ``isChampion``, ``isBestSeller``, ``isLicensed``).
- **Archival:** products are soft-archived (``status: ARCHIVED``) rather than hard-deleted, preserving historical hold/sale references.
- **Sizing:** size lists are derived automatically from subcategory/age-group/category via ``getDefaultSizes()``, with sizeless categories (accessories, mugs, bobbleheads) bypassing size selection entirely.
- **Player linkage:** products can be mapped to one or more ``Player`` records via ``PlayerProduct``, powering the Popular Players shoppable-gear experience.

22. Sales & Clearance Management

Sales & Clearance is implemented as a unified catalog (`isClearance` flag) with subcategory-based organization, avoiding duplicate product entries while still enabling focused browsing by audience.

- Subcategories: **Men, Women, Kids, Accessories** — assigned by admins at the point a product is marked Sale/Clearance.
- Fans can filter the unified Sales & Clearance catalog by subcategory without the catalog needing separate duplicate listings per audience.
- The product detail page automatically surfaces a **Target Audience** label (Men/Women/Kids) for clearance/sale items, inherited directly from the product's category/subcategory — no duplicate data entry required, and hidden entirely when not applicable.
- Filtering is fully responsive, using the same client-side filter pattern as the main shop catalog, consistent across desktop and mobile.

23. Brand and Category Management

Category taxonomy is centralized in a single constants module so every screen — storefront, admin, and reporting — reads from one authoritative list, preventing category drift between areas of the application.

Category	Purpose	Structure
men / women / kids	Audience categories	Full subcategory sets (Jerseys, Tees, Hoodies, Hats, etc.)
accessories	Non-apparel merchandise	Bags, drinkware, collectibles
featured	Curated cross-category spotlight	Flag-driven (isFeatured)
new-arrivals	Recently added merchandise	Flag-driven (isNewArrival)
sales-clearance	Unified discount catalog	Men / Women / Kids / Accessories subcategories
sports	General sport-branded merchandise	Flag-driven (isSport)
authentication	Authenticated/licensed memorabilia	Flag-driven (isAuthenticated)
blank-jersey	Player-name/number-free jerseys	Men / Women / Kids, filterable by Brand

Brands are managed as first-class records (`Brand` model) via `/admin/brands`, each with a name, logo, and optional description, and every product references a brand for filtering on brand pages and within category views (including the new Blank Jersey category's brand-based filtering).

24. Reporting and Analytics

Report	Location	Content
Operational Overview	/admin/analytics	Active holds, low-stock alerts, today's pickups, at-a-glance KPIs
Sales Reports	/admin/reports	Revenue trends, top-selling products, weekly/period comparisons
Hold Conversion	/admin/reports	Pickup vs. release vs. expiry rates, no-show tracking
Inventory Health	/admin/inventory/history	Stock levels by location, transaction history, transfer trail
Engagement Analytics	/admin/analytics	Product likes/favorites trends, review volume
Audit Trail	/admin/audit-log	Full chronological record of every sensitive action, filterable by actor/action

Reports support CSV export for offline analysis and are backed by short-lived in-memory caching to keep dashboards responsive under repeated polling without adding database load.

25. Security and Access Control

- **Authentication:** NextAuth Credentials provider with bcrypt-hashed passwords; JWT sessions expire after 8 hours.

- **Authorization:** centralized `requireRole()` helper checks the caller's role against the `actionMinRole` map before any protected route executes its logic.
- **API-key auth (POS):** POS webhook endpoints authenticate via dedicated `PosApiKey` records, independent of admin sessions, scoped to transaction ingestion only.
- **Rate limiting:** a sliding-window limiter protects admin API paths (default 100 requests/minute) against abuse or runaway automation.
- **Cron protection:** the hold-expiry cron endpoint requires a Bearer `CRON_SECRET` header, preventing unauthorized manual triggering.
- **Auditability:** every sensitive create/update/delete records an `AuditLog` entry capturing actor, action, entity, and before/after state.
- **Data isolation:** `tenantId` on core models establishes tenant boundaries in preparation for multi-tenant/multi-store expansion.

26. Performance and Scalability Considerations

- Database transactions scope hold creation/resolution and inventory transfers tightly to minimize lock contention under concurrent load.
- In-memory caching (~30s) on analytics endpoints absorbs dashboard polling without repeated heavy aggregation queries.
- Indexed lookups (e.g., `isBlankJersey`, `isClearance`, `isFeatured` flags; phone number on `Customer`) keep catalog and hold-lookup queries fast as data volume grows.
- The tenant-aware schema and multi-location `StoreLocation` model allow the platform to scale horizontally to additional venues without a schema rewrite.
- Next.js server components and route-level static/dynamic rendering choices (`force-dynamic` on API routes) balance freshness needs against build-time performance.

27. API and Data Flow Architecture

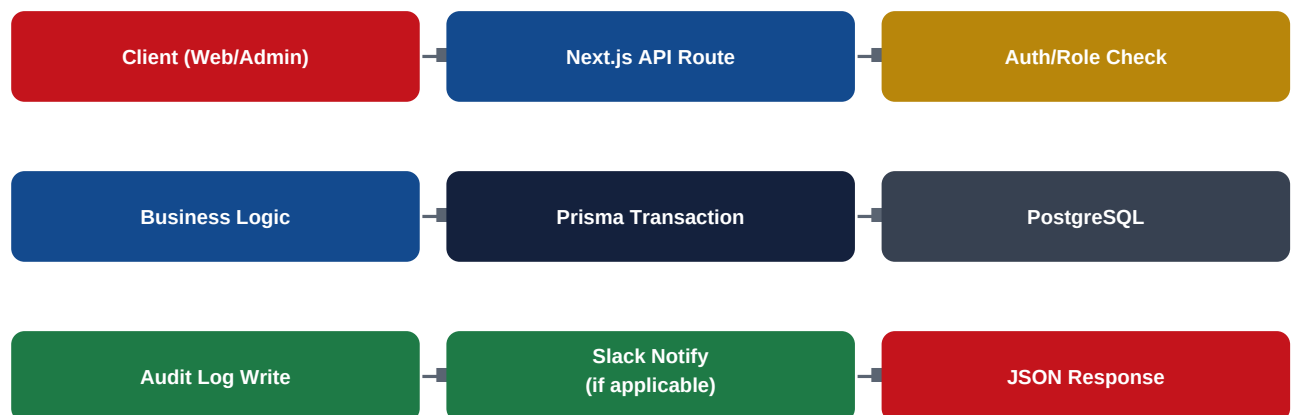


Figure 27.1 — Typical request/data-flow path through a protected admin API route

Public routes (catalog, product detail, players, holds creation/lookup, feedback, reviews) require no authentication and are optimized for fast anonymous access. Admin routes uniformly pass through session validation, role authorization, business logic (often wrapped in a Prisma transaction), audit logging, and — where relevant — a Slack notification side-effect, before returning a JSON response.

28. Business Rules and Validation Logic

- A customer may hold a maximum of **3 active reservations** at any time.
- Section 123 (stadium queue) pickup is only offered/honored on an active **game day**; otherwise the request is rejected with a clear message.
- The extended 48-hour hold window applies only to standard (non-stadium) holds, only when enabled in ``HoldSettings``, and never on a game day.
- Inventory can never go negative — hold creation and POS sales re-validate live availability inside the same transaction that commits the change.
- Products are never hard-deleted while referenced by historical holds/sales — archival (``status``) preserves referential integrity.
- POS events are ingested idempotently by ``externalId``, preventing duplicate transaction application on retry/replay.
- Target-audience and Blank Jersey subcategory labeling is always derived from ``category`/`subcategory``, never a separate duplicated field, guaranteeing consistency.

29. UI/UX Design Principles

- **Mobile-first:** every public and admin surface is designed for small-screen usage first, then progressively enhanced for tablet/desktop.
- **Brand-consistent palette:** Jays Navy, Royal, Red, and Ice tones are reused across badges, buttons, and highlights rather than introducing new colors per feature.
- **Minimal noise:** redundant labels and low-value copy (e.g., generic “In Stock” text, verbose field labels) are removed in favor of direct, glanceable information.
- **Consistent card language:** hover elevation, scale, and shadow transitions are shared across product cards, player cards, and related-item cards.
- **Progressive disclosure:** compact layouts (e.g., the condensed Upcoming Match card) show only what's essential, expanding detail only where users need it.
- **Accessibility of key actions:** primary actions (Feedback button, Hold CTA) are positioned prominently with high-contrast, brand-appropriate coloring.

30. Error Handling Strategy

- API routes return structured JSON error payloads with descriptive codes (e.g., ``SECTION_123_GAME_DAY_ONLY``) rather than opaque failures, enabling precise client-side messaging.

- Hold creation/resolution failures roll back the entire Prisma transaction, guaranteeing inventory and hold state never diverge on partial failure.
- Public-facing forms (hold creation, feedback, reviews) validate input both client-side (immediate feedback) and server-side (authoritative enforcement).
- Admin routes return standardized 401/403 responses for authentication/authorization failures, distinct from 4xx validation errors.
- Cron and webhook endpoints log failures for operational visibility without exposing internal error detail to external callers.

31. Future Enhancements and Version Roadmap

- Native account/loyalty layer building on the existing phone-identified customer model.
- Expanded Player feature: player-specific promotions, stats-driven merchandising, and social sharing.
- Additional stadium sections/queues beyond Gate 5 and Section 123 as operational needs grow.
- Deeper analytics: predictive low-stock alerts and demand forecasting tied to the game-day calendar.
- Multi-venue/multi-tenant activation using the existing ``tenantId`` foundation already present in the schema.
- Expanded notification channels (SMS/email) alongside the existing Slack integration.

32. Testing Strategy

- **Type safety:** ``tsc --noEmit`` run as a required gate before merging any change, backed by Prisma's generated types across the entire data layer.
- **Linting:** ESLint (``eslint-config-next``) enforces consistent code style and catches common React/Next.js pitfalls.
- **Manual verification:** feature changes are verified against real admin/storefront flows (create/edit/archive, hold create/resolve/expire) before release.
- **Data integrity checks:** transactional hold and inventory logic is manually validated against edge cases (partial pickup, concurrent holds, game-day boundary conditions).
- **Recommended next step:** introduce automated integration tests around the hold lifecycle and inventory ledger, given their correctness-critical nature.

33. Deployment Architecture



Figure 33.1 — Deployment pipeline

- Hosted on **Vercel**, with ``prisma generate`` wired into both the build and postinstall scripts to guarantee the client is always current in serverless environments.

- All API routes are explicitly marked ``force-dynamic`` to prevent Prisma client initialization from occurring during static build analysis.
- A daily Vercel Cron job (``0 0 * * *``, aligned to Hobby-plan limits) triggers the hold-expiry job in production.
- Environment-based configuration (``CRON_SECRET``, database URL, NextAuth secret, Slack webhook, POS keys) keeps sensitive credentials out of source control.

34. Maintenance and Support Plan

- The ``AuditLog`` provides first-line diagnostic capability for any operational question (“who changed what, and when”) without needing direct database access.
- Centralized constants (``constants.ts``) and permission maps (``authorize.ts``) mean taxonomy or role changes are made in one place and propagate consistently.
- Soft-archival conventions (products, players) protect historical data integrity, simplifying support investigations into past holds/sales.
- Scheduled game-day data entry is the only recurring manual maintenance task tied to core business rules; all downstream behavior (Section 123 eligibility, hold windows, landing-page display) updates automatically.

35. Project Risks and Mitigation Strategies

Risk	Mitigation
Overselling across channels	Shared SizeInventory ledger + transactional writes across web, hold, and POS eliminate channel drift.
Stale/expired holds blocking inventory	Dual expiry mechanism: daily cron plus inline auto-expire on every read path.
Timezone-driven date mismatches	Game-day date/time handled and displayed consistently in UTC across admin entry and landing-page rendering.
Unauthorized admin actions	Centralized role-based authorization enforced uniformly on every protected route.
Duplicate POS transaction application	Idempotent ingestion keyed on POS <code>`externalId`</code> .
Feature creep diluting taxonomy consistency	All category/flag definitions centralized in one constants module referenced everywhere.

36. Conclusion and Long-Term Vision

Version 2.0 of Jays Shop represents a mature, tightly integrated retail operations platform: one inventory ledger spanning web, hold, and point-of-sale channels; one taxonomy governing every category and merchandising flag; one role-based permission model protecting every admin action; and one audit trail giving full operational transparency. The platform's architecture — centralized constants, transactional business logic, and a consistent API/UI pattern language — means new retail capabilities (additional pickup sections, new merchandising categories, deeper player integrations, expanded reporting) can be layered on without destabilizing what already works.

“A connected retail platform where every browse, reservation, and pickup reflects the same standard of excellence fans expect on game day.”

This document should be treated as a living reference: as the platform evolves toward Version 3.0 and beyond, each new capability should be added to the relevant section here, keeping this a single, trustworthy source of truth for how Jays Shop works and why it was built this way.